CSA10 – SOFTWARE ENGINEERING

Assignment 2: Requirements and Scalable Architecture for the Omnichannel Order Fulfilment Platform ("FulfilIQ")

Student name	**Ashira Bridget Susan S**	**Register number**	**192524316**
Programme / section	**CSA1021**	**Submission date**	**11/11/2026**

CO2: Analyze software requirements to design scalable and efficient system architectures using collaborative tools (Jira, Confluence) and structured modelling (UML, architectural patterns). **Flow:** Investigate → Specify → Model → Architect → Validate → Redesign → Defend.

Evidence statement. Stakeholder responses are **simulated** and labelled as such. Results marked "model" come from an executable in-memory Python model (**fulfiliq_test_model.py**, delivered with this report) run on synthetic data; it demonstrates rule logic only, not database concurrency, performance, live inventory control or secure payment handling. All workload figures are **planning assumptions**, not measured capacity. The refund limit, tax rate and expiry length are illustrative.

Q1. Stakeholder Needs and Conflict Resolution

1.1 Investigation questions

Method: a structured process review plus a short questionnaire to six role representatives. Real access was unavailable, so the answers below are **simulated responses (SR)**, written to be plausible and to expose requirement conflicts.

#	Question (unresolved requirement)	Asked of	Simulated response (SR) and finding
Q-A	How does a picker learn that a unit shown on the pick list was taken by another buyer?	Picker	SR: "Sometimes I walk to the shelf and it is gone." Finding: reservations must be made at order time, not at pick time (→ FR04).
Q-B	Who may approve a refund above the standard limit, and can a support agent do it to keep the customer happy?	Support, Finance	SR (Finance): "Anything above the limit needs us, with a name and time." SR (Support): "Waiting hurts satisfaction." Finding: conflict C4.
Q-C	Which order details may a support agent see, and may they see payment data?	Support, Admin	SR: "Status, items and a phone number; never card or payment details." Finding: masked view (→ FR12).

#	Question (unresolved requirement)	Asked of	Simulated response (SR) and finding
Q-D	If the order is stored but the courier shipment request fails, who is responsible and what does the customer see?	Admin, Customer	SR: "The system must retry; do not make me order again." Finding: durable outbox with "Shipment pending" status (→ FR11).
Q-E	What should happen if the gateway sends the same payment confirmation twice, or after the cart has expired?	Admin, Store Manager	SR: "Never charge or ship twice; if we sold the unit to someone else, refund." Finding: FR06, FR07.

1.2 Stakeholder analysis

Stakeholder	Primary need	Influence	Treatment in the specification
Customer	Fast checkout, clear tracking, own data only	Medium	FR02, FR12; NFR01, NFR06. "Free instant refund" request declined (see C4).
Picker	Accurate pick lists, reliable reservation, scanning even when offline	Medium	FR04, FR09, FR19. Request to "edit stock freely" declined: corrections need a recorded reason.
Support agent	Readable order/return status without payment data	Medium	FR12, FR13. Request to "approve small refunds myself" limited by the finance limit.
Finance reviewer	Auditable approval before refunds above limit	High	FR14, FR15, FR16.
Store manager	Consolidated sales, stock and refund reports	Medium	FR17 (summary tables, Q6).
Administrator	Regional configuration, user management, traceability for engineers	High	FR01, FR18, FR16.

Not every request becomes a requirement: "let customers see all courier internals", "automatic refunds for any complaint" and "live stock on a public API" were rejected as out of scope or as violating privacy/accountability.

1.3 Conflicts and resolutions

Conflict	Resolution and justification	Acceptance implication	Spec impact
C1 Checkout speed vs stock accuracy	Reserve stock inside the same transaction as the order, with a 15-minute expiry. A few extra milliseconds are accepted because overselling creates refunds and lost trust that cost more than latency.	Two buyers, one unit: one order succeeds, other is told "out of stock".	FR04, FR05, NFR01
C2 Support visibility vs customer privacy	Support sees status and masked contact only; card data is never stored; role is taken from the authenticated server session, not a UI selector.	Cross-customer read and role spoof are rejected.	FR01, FR12, NFR05
C3 Instant payment confirmation vs availability	Commit order locally as Placed ("Payment pending"); mark Paid only on a signed, verified callback; browser redirect is not proof.	Lost or repeated callback never loses the order or shows Paid early.	FR06, FR11
C4 Fast refunds vs finance control	Agents/customers may request; refunds above the limit need finance approval; execution is a separate, approval-gated step.	Non-finance approval rejected; execute without approval rejected.	FR14, FR15, FR16

1.4 System boundary, actors and exclusions

- **Inside the boundary:** storefront/staff web UI, REST API, persistence, regional policy rules, integration worker, reporting module.
- **Actors:** Customer, Picker, Support Agent, Finance Reviewer, Store Manager, Administrator (human); Payment Gateway, Courier API (external systems, outside the permission model); warehouse scanner (input device).
- **Excluded from the representative release:** autonomous refund decisions, machine-learning demand forecasting (low-stock alerts are a fixed deterministic threshold), public API access, real card handling, live gateway/courier access (test stubs used and disclosed).

Q2. Prioritized and Testable Requirements Specification

2.1 Tool arrangements compared and selected

Criterion	A: Jira + Confluence	B: GitHub Issues/Projects + Markdown specification in repo	C: Linear/Notion + draw.io files
Traceability	Issue links; Confluence macros embed issue lists	Issue/PR links, commit-linked specs; strong with code	Manual cross-links; weaker
Collaboration / history	Comments, page versions, approvals	Pull-request review, full Git history	Good comments; limited diff history
UML support	Marketplace plugins (draw.io, PlantUML)	Mermaid/PlantUML rendered from text, diffable	draw.io files, not diffable
Accessibility / cost	Browser-based; free tier ≤10 users	Free; text-first, screen-reader friendly	Free tiers; export limits

Selection: A (Jira + Confluence) for requirements and decisions, with diagrams authored as text (PlantUML/Mermaid/Graphviz) and stored in GitHub. Justification: the course reference stack, issue-to-page traceability and approval history serve the six stakeholders, while text-based diagrams give version diffs. **Limitations and mitigations:** the free tier is limited to ten users (small team acceptable); Confluence diagram macros can drift from the repository, so diagram source files are the single source of truth and pages embed rendered exports tagged with the commit; Jira Cloud needs internet access, so a weekly export of requirement records is kept in the repository.

Prioritization method: MoSCoW adjusted by **customer-harm risk** (financial loss, privacy exposure, or inconsistent state raises a requirement one level). Hence FR04, FR06, FR14 and FR01 are Must even though customers do not ask for them directly. Each requirement is a Jira issue (key FIQ-n) with fields: ID, source, priority, acceptance criteria, linked scenario.

2.2 Functional requirements (FR) – Rev 1 (FR01–FR17) and Rev 2 additions (FR18–FR20)

ID	Requirement	Source	Pri.	Acceptance criterion
FR01	Authenticate users; authorize every command by server-side role	C2, Admin	Must	A support or customer session calling approve-refund gets HTTP 403; changing a UI role field changes nothing.
FR02	Place order with validated fields	Customer	Must	Quantity ≤ 0 or unknown SKU rejected; client-supplied price ignored.
FR03	Compute total = server price × qty + regional tax	Finance	Must	Test table: 2 × 100.00 at 18% ⇒ 236.00.
FR04	Reserve stock atomically with order creation	C1, Picker	Must	20 concurrent buyers of 1 unit ⇒ exactly 1 order; available never < 0.
FR05	Release unpaid reservation after 15 min (once)	C1	Should	At t=16 min the order is Cancelled, reservation released once, audit row written.
FR06	Confirm payment only from a verified callback; idempotent on (gateway, event_id)	C3	Must	Same event ×10 ⇒ 1 payment, 1 Paid transition, 1 shipment intent; bad signature/amount rejected.
FR07	Handle late payment after expiry	Q-E	Must	If unit resold: order not Paid, flagged "needs refund", refund request opened.
FR08	Enforce order lifecycle Placed→Paid→Packed→Dispatched→Delivered→Closed; Cancelled only before payment	Admin	Must	Placed→Dispatched rejected; no state change on rejection.
FR09	Pick list shows only paid, reserved items; picker confirms pick	Picker	Should	Unpaid order absent from list; pick sets Packed.
FR10	Dispatch within 24 h (UTC) of payment; requires payment + pick	Manager	Should	Dispatch without pick rejected; orders unshipped at 24 h flagged on dashboard.
FR11	Synchronize courier/gateway through a durable outbox with reconciliation	C3, Q-D	Must	Courier timeout ⇒ order stays Paid, intent Pending; retry reuses correlation key ⇒ 1 shipment.

ID	Requirement	Source	Pri.	Acceptance criterion
FR12	Customer tracking; support sees masked order view	C2	Should	Customer reads only own orders; support view has masked phone, no payment fields.
FR13	Return request only for eligible (Dispatched/Delivered) orders	Support	Should	Request on a Placed order rejected.
FR14	Refund above limit needs finance approval; approval ≠ execution	C4	Must	Approve by non-finance rejected; approver identity and time stored.
FR15	Execute approved refund; failed execution keeps approval and can be retried	Finance	Must	Gateway failure ⇒ Failed; retry ⇒ Executed, order Closed; unapproved execute rejected.
FR16	Audit every accepted command (actor, time, version) in the same transaction	Admin	Must	One audit row per accepted command; none for rejected ones.
FR17	Consolidated sales, stock and refund reports	Manager	Could	Totals reconcile with order table (difference 0) for test data.
FR18 (R2)	Region-scoped tax/shipping/refund rules, versioned	Admin	Must	Order stores policy_version; changing a rule does not alter past orders.
FR19 (R2)	Offline scanner buffers uniquely identified events; server dedups on replay	Picker	Must	Replay of 5 events twice ⇒ stock changes 5 times only.
FR20 (R2)	Locally queued refund approval displays “Pending server validation”	Finance	Must	UI never shows Approved until server state is Approved.

2.3 Non-functional requirements (NFR) – measurable

“Checkout should be fast” is replaced by NFR01 (operating condition + observable threshold).

ID	Quality	Requirement (condition → threshold)	Source / acceptance
NFR01	Performance	With 50 concurrent synthetic users, 1,000-SKU dataset, 100 ms network RTT, 1-min warm-up and 5-min run, p95 checkout latency ≤ 800 ms and error rate < 1%.	Customer; load script, recorded hardware
NFR02	Availability	Order-read and callback endpoints available ≥ 99.5% per month (planned maintenance excluded); a courier outage must not reject callbacks.	Admin; uptime log, outage drill
NFR03	Consistency	Zero negative stock and zero duplicate payments in 1,000 repeated concurrent/duplicate trials.	Finance; invariant check
NFR04	Recovery	RTO ≤ 15 min and RPO ≤ 5 min after database restore in a test environment.	Admin; restore drill
NFR05	Security	All 6 roles tested for permitted and forbidden critical actions; no card number stored; callbacks signature-verified.	C2; role matrix test
NFR06	Usability	5 of 5 test users complete purchase + tracking by keyboard only in ≤ 3 min; status shown as text, not colour alone.	Customer; usability script
NFR07 (R2)	Scalability	Planned 500 simultaneous users and 1,000 events/min (design target); report queries must not raise checkout p95 above 800 ms.	Manager; Q6 validation

2.4 Order rules, assumptions and open questions

- **Order rules:** total = server price + regional tax; one owner per order; allowed statuses as FR08; dispatch deadline 24 h after payment (UTC); repeated gateway event returns the existing result; refunds above the limit need a finance reviewer; stock reserved = on-hand − available.
- **Assumptions:** A1 refund limit = 100.00 (illustrative); A2 reservation expiry 15 min; A3 tax 18% flat in the model; A4 three regions initially; A5 synthetic data only; A6 the gateway signs callbacks and may resend them.
- **Unresolved questions:** UQ1 true refund limit per region; UQ2 best expiry length; UQ3 regional return windows; UQ4 retention period for customer addresses.
- **Conflict influence:** C1 produced FR04/FR05 (reservation with expiry, accepting extra latency in NFR01); C4 produced FR14/FR15 (approval-gated, separately modelled refund states) and FR20 in Rev 2.

Q3. UML Models and Traceability

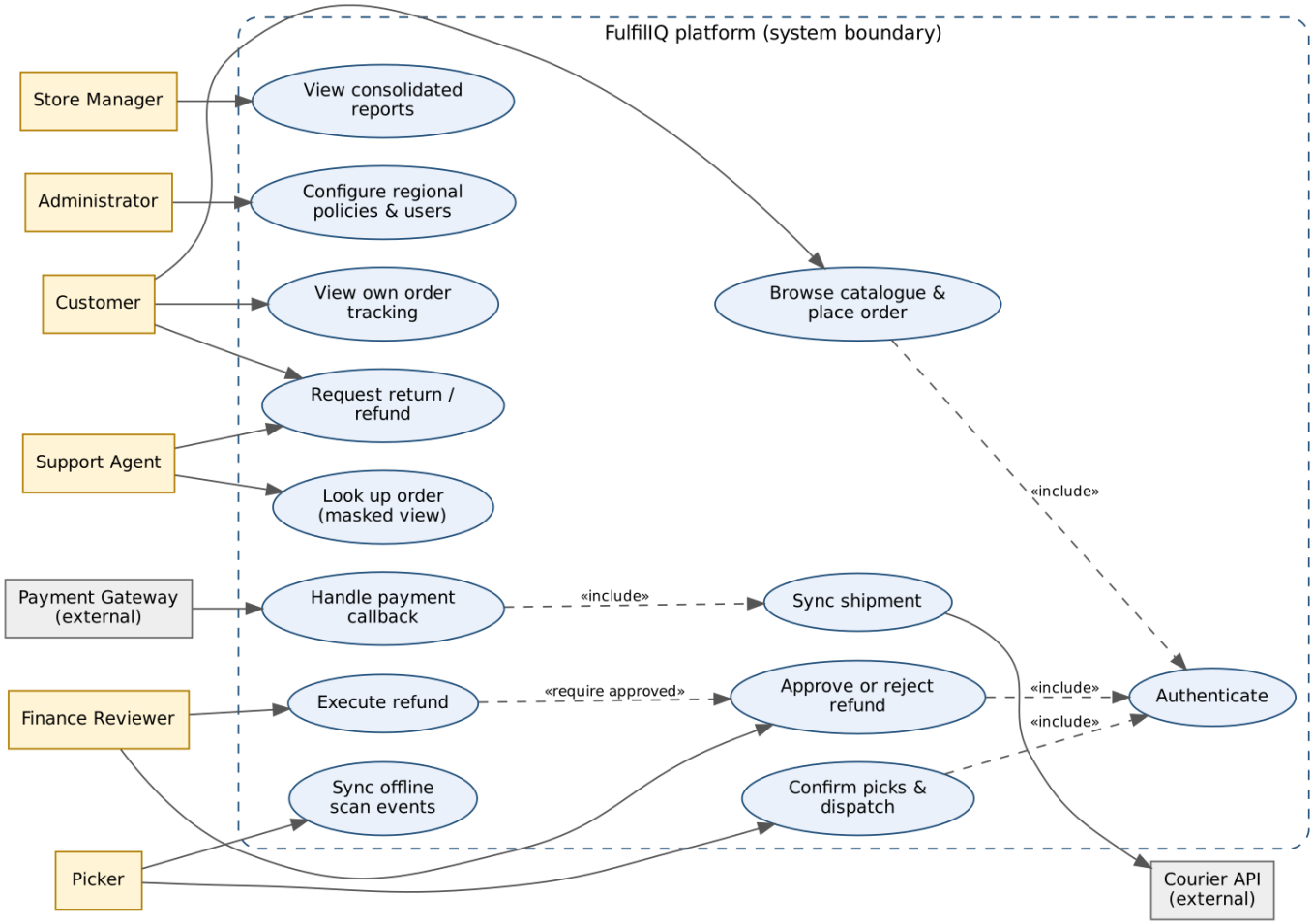


Figure 1. Use case diagram – actors and principal interactions (refund approval and execution are separate use cases).

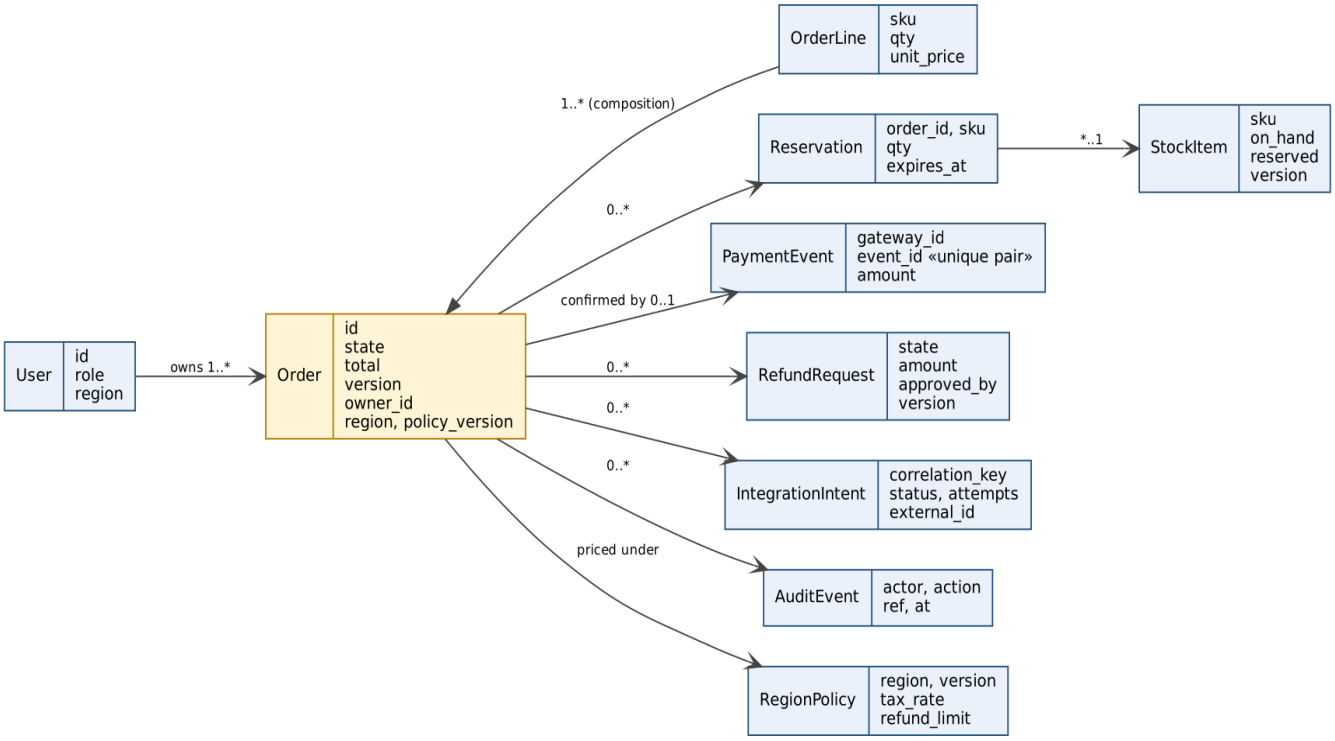


Figure 2. Domain class diagram (core entities and relationships).

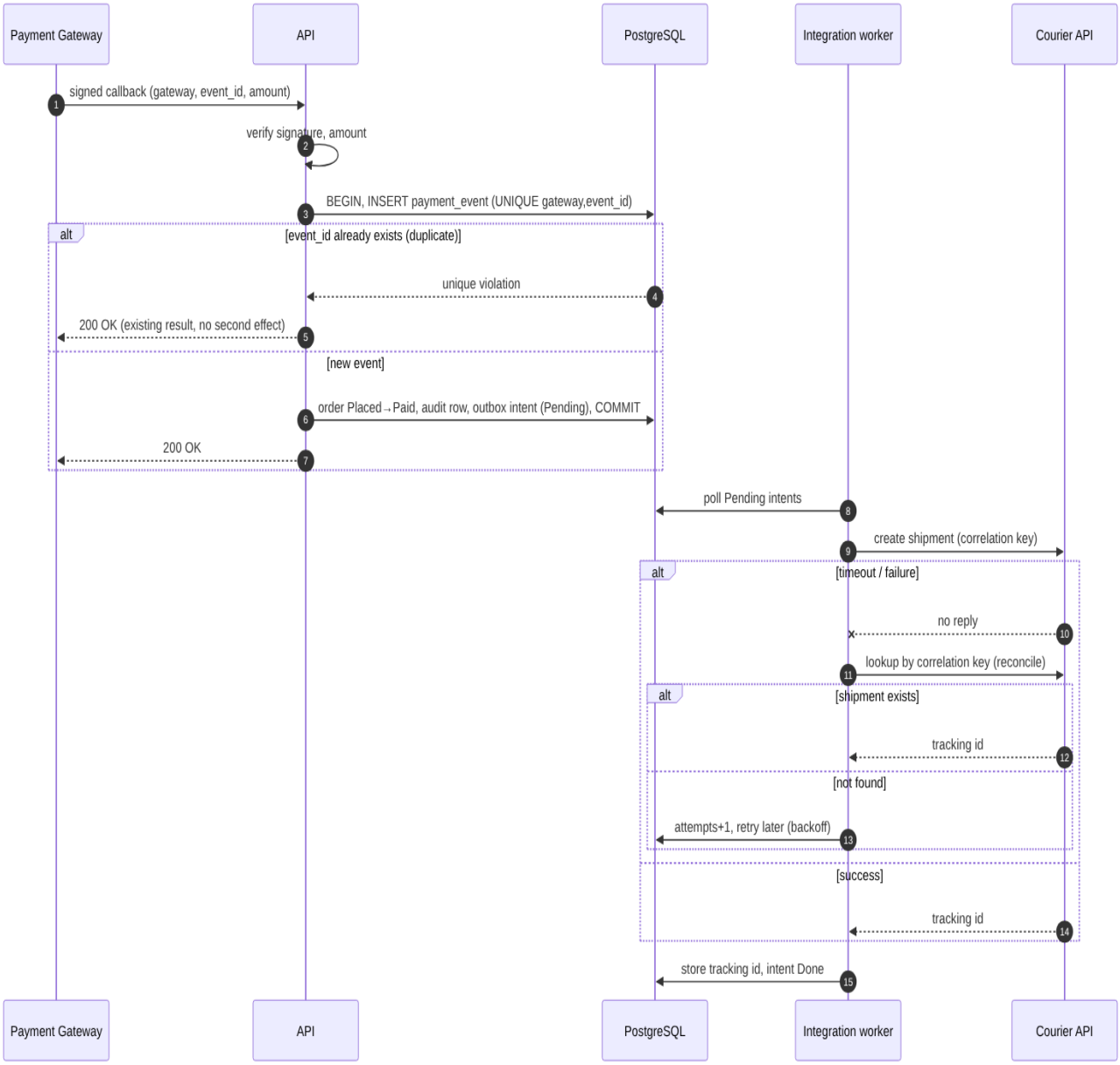


Figure 3. Sequence: payment callback → order confirmation → shipment creation → courier timeout, reconcile and retry.

Note on Figure 3: the callback response does not wait for the courier. The payment event, the Paid transition, the audit row and the shipment intent commit in one transaction; the worker later sends the shipment with a correlation key and, after a timeout, looks the shipment up before retrying so that no second shipment is created.

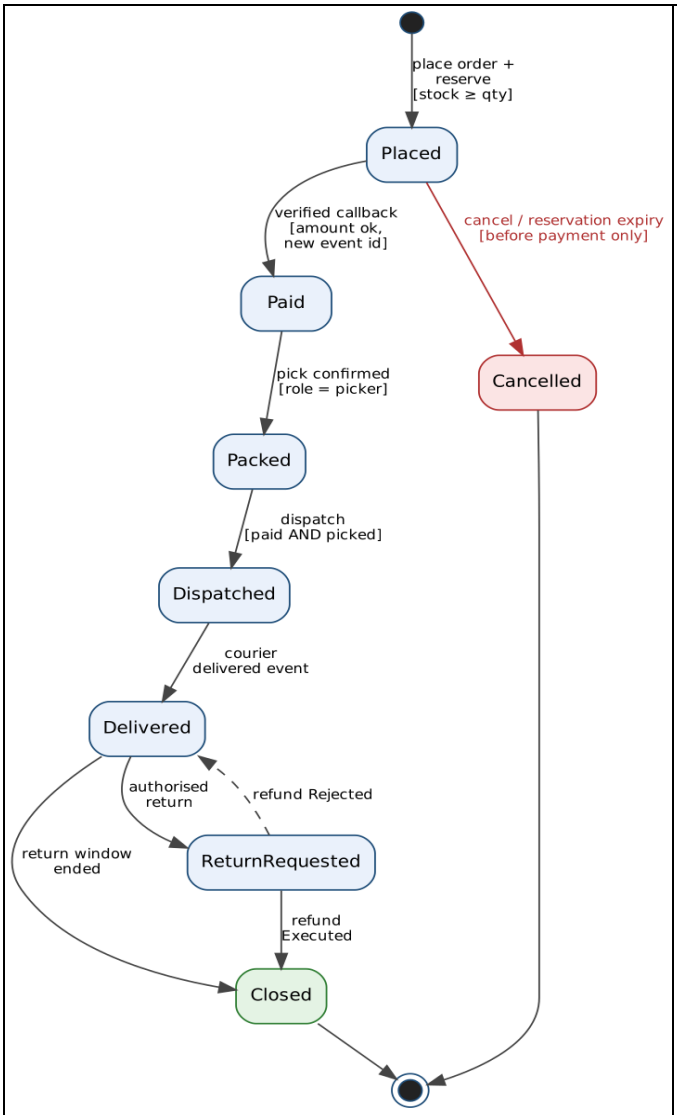


Figure 4. Order state machine

Guards on every transition: the command checks (1) the authenticated role, (2) the expected version (optimistic locking; a stale version is rejected, not overwritten), (3) required evidence, and (4) the current state.

Placed → Paid only through a verified callback with a new event id and the correct amount. **Paid → Packed** needs a picker. **Packed → Dispatched** needs payment and pick confirmation. **Cancelled** is reachable only from Placed (manual cancel or reservation expiry). A delivered order with an authorised return closes only after the refund is **Executed**; a rejected refund returns it to Delivered.

Courier states and warehouse-board states are mapped to these order states, not assumed identical (e.g. courier "out for delivery" does not change the order state).

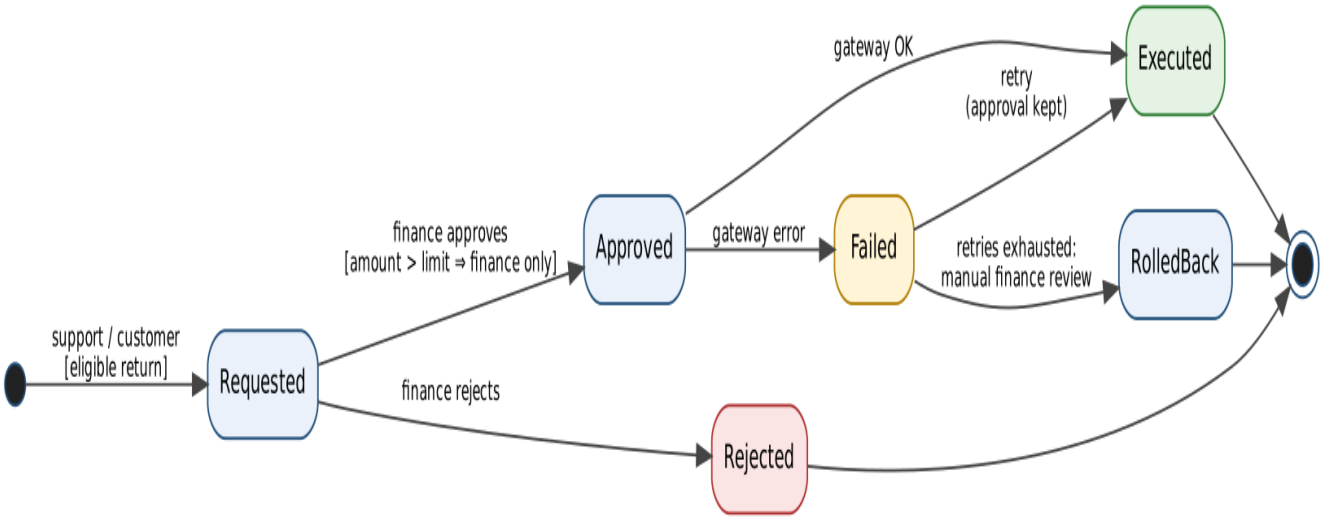


Figure 5. Refund state machine – approval, rejection, execution, failure, retry and manual rollback.

3.1 Traceability matrix (Requirement → UML element → Component → Validation)

Req.	UML element	Architectural component	Validation scenario
FR01	Use case "Authenticate"; role attribute on User	Auth & RBAC module	T6 (unauthorised approve, cross-customer read)
FR03	Order.total, RegionPolicy.tax_rate	Order module + Regional policy	T11 (price tampering, tax 236.00)
FR04	Reservation, StockItem (Fig. 2)	Stock module + DB row lock	T1 (last unit, 20 buyers)
FR06	PaymentEvent unique identity; Fig. 3 steps 3–6	Payment module, UNIQUE constraint	T2 (duplicate callback ×10)
FR07 / FR05	Fig. 4 Placed→Cancelled; Reservation.expires_at	Stock module + scheduler	T7 (late payment after expiry)
FR08 / FR10	Order state machine (Fig. 4)	Order command service	T4 (dispatch without payment/pick)
FR11	IntegrationIntent; Fig. 3 retry branch	Outbox worker	T5 (courier timeout then retry)
FR14 / FR15	RefundRequest; Fig. 5	Refund module + gateway adapter	T3 (refund fails, retry), T6
FR19	Scan event (node_id, event_id)	Ingest queue + dedup index	T8 / S1 (replay burst)
FR20	Fig. 5 Requested state; UI "Pending"	Refund module + React UI	T9 / S2 (queued approval)

3.2 Inconsistencies found while connecting the models and their corrections

#	Inconsistency	Correction
I1	The first use-case draft showed "Customer sees Paid" after the browser success redirect, while the sequence diagram confirms payment only from the gateway callback.	Payment is confirmed only by the verified callback; the UI shows "Payment pending" until then (FR06; Fig. 3).
I2	The class model had a role on User, but a draft UI mock-up let a staff member "switch role", contradicting FR01.	Identity and role come from the authenticated server session; the role selector was removed and T6 tests the spoof.

#	Inconsistency	Correction
I3	The refund state model had Failed with no outgoing edge, which would strand an approved refund and contradict scenario T3 (retry).	Added Failed → Executed (approval kept) and Failed → RolledBack for manual finance review (Fig. 5).

Q4. Alternative Architectures and Selected Design

Dimension	Alternative A: Modular monolith + integration worker	Alternative B: Microservices + event broker
Components	Auth, Order, Stock, Payment, Refund, Reporting modules in one FastAPI app; PostgreSQL; outbox worker; React UI.	Separate Order, Inventory, Payment, Refund, Notification and Reporting services; Kafka/RabbitMQ; API gateway.
Communication / data ownership	In-process calls; one database, each module owns its tables.	Async events + REST; each service owns its database; sagas for cross-service flows.
Duplicate payments	UNIQUE(gateway, event_id) in the same transaction as the Paid transition.	Idempotent consumer in Payment service plus de-duplication of broker redelivery.
Overselling last unit	Conditional update / row lock on stock inside the order transaction.	Inventory service serialises reservations; Order waits for a “Reserved” event or compensates.
Scalability	Replicate stateless API/worker; DB is the shared bottleneck.	Scale each service independently; higher ceiling.
Consistency / availability	Strong consistency; central DB failure stops writes.	Eventual consistency; partial failures are isolated but states can diverge.
Security	Single trust boundary; one place for RBAC.	More endpoints and secrets; service-to-service auth needed.
Operational complexity	Low: 3 containers; small team can run it.	High: broker, 6+ deployments, tracing, saga repair.
Likely failure points	DB, worker backlog, large releases.	Broker lag, lost/duplicated events, distributed debugging.
Maintenance	Module boundaries must be enforced by code review.	Contract versioning across teams.

Selection: Alternative A. The central requirements (FR04, FR06, FR14–FR16) are consistency problems for which a single transaction is the simplest correct mechanism, and the small team cannot yet justify the cost of B (stated constraint). B is retained as a later option once measured, independent load (e.g., reporting or stock) justifies splitting.

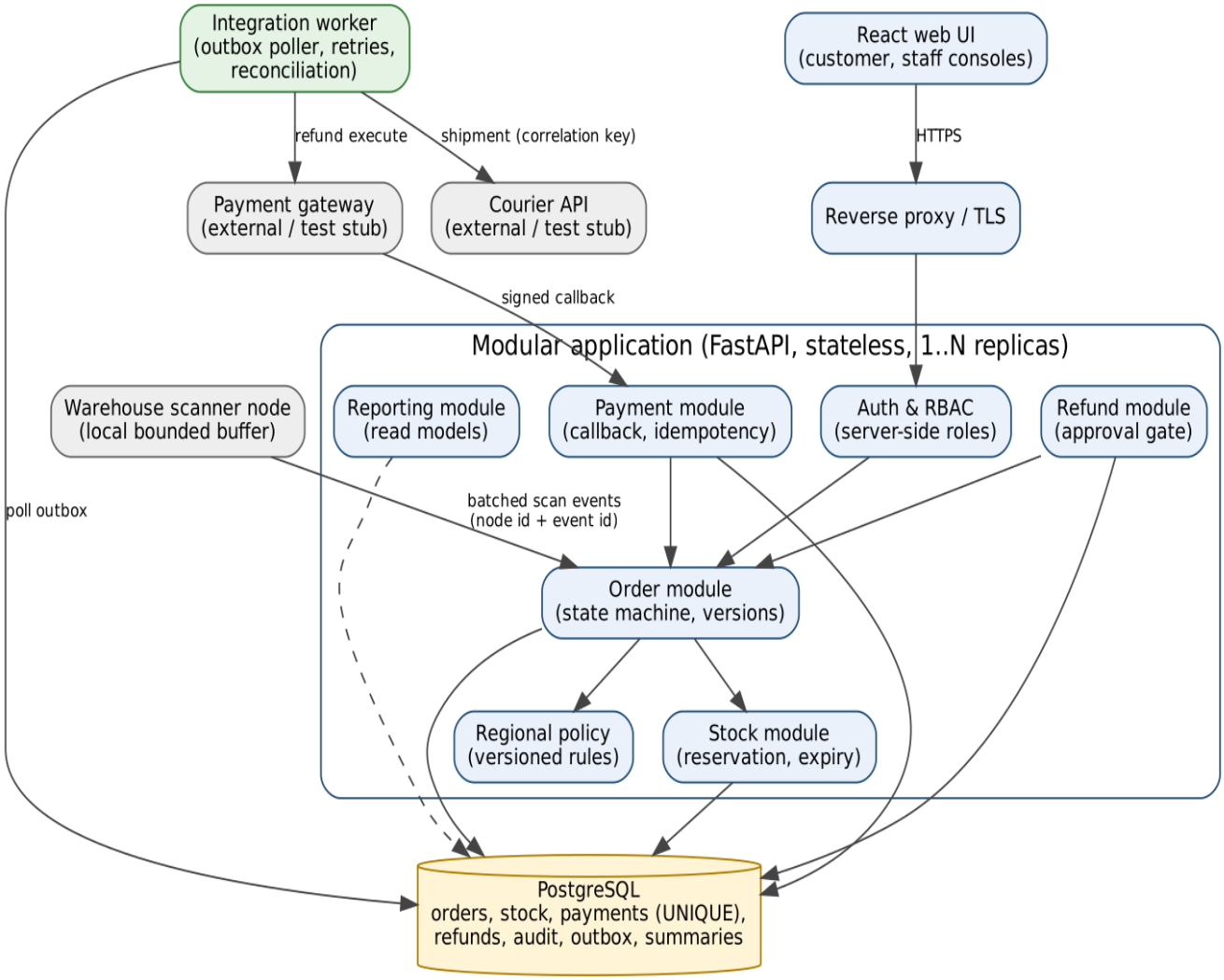


Figure 6. Selected architecture – component diagram.

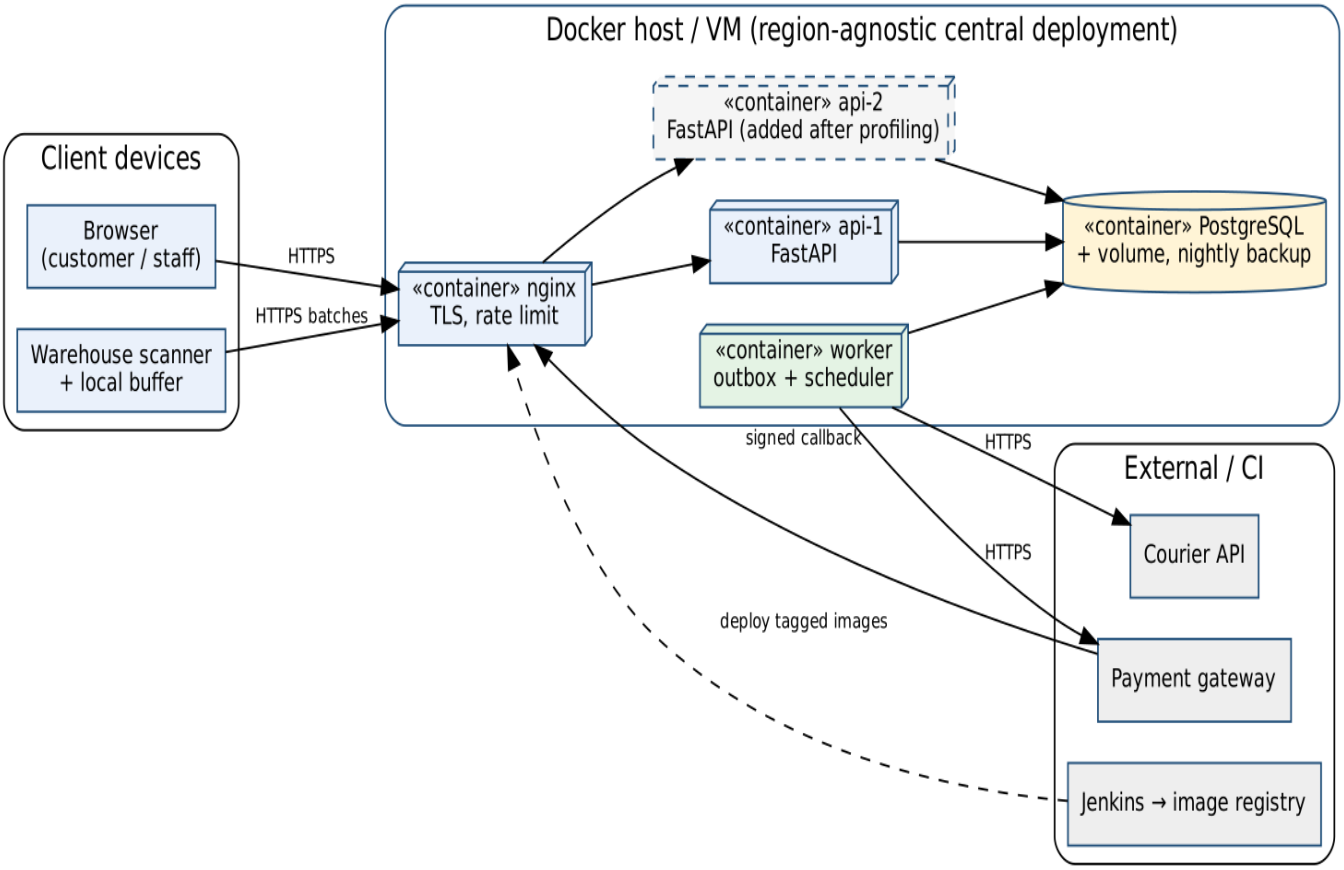


Figure 7. Selected architecture – deployment diagram for the bounded initial release (second API replica added only after profiling).

4.1 Architectural decision records

ADR	Problem	Alternatives	Decision and justification	Consequences
ADR 01	Order, stock, refund approval, audit and intents must change together.	Relational DB vs document store per service.	PostgreSQL: transactions, UNIQUE and foreign-key constraints match the invariants; a document store could also do transactions, so the choice is about schema and operational fit.	Schema migrations needed; DB is a single point of failure (mitigated by backups, NFR04).

ADR	Problem	Alternatives	Decision and justification	Consequences
ADR 02	Order must survive gateway/courier outages (C3).	Synchronous call in request; broker; DB-backed outbox.	Transactional outbox + worker: intent is committed with the order; the worker retries with correlation key and reconciliation.	Delayed sync, a worker to monitor, status "Shipment pending" visible to users.
ADR 03	Support/customer must not gain finance rights; stale writes must not overwrite state.	UI-based role hiding; per-service tokens; server RBAC + version checks.	Server-side RBAC from session, idempotency keys and optimistic version column on Order and Refund.	Clients must handle HTTP 409 conflicts; role matrix must be tested (NFR05).

Q5. Critical Behaviour Validation and Design Revision

5.1 Invariants and operation contracts

- **INV1:** one (gateway, event_id) creates at most one payment and one order confirmation. **INV2:** a refund cannot be Executed unless it was Approved (approved_by set).
- **INV3:** available stock = on_hand − reserved is never below zero. **INV4:** every accepted order command writes exactly one audit row in the same transaction. **INV5:** a stale version never silently overwrites accepted state.

Operation	Preconditions	Postconditions
placeOrder(session, sku, qty)	Authenticated customer; qty > 0; available ≥ qty.	Order Placed; reservation of qty created once; total computed on server; audit appended. On failure: no state change.
confirmPayment(gateway, event, order, amount)	Signature valid; order known; amount equals total; event unseen (or seen).	Unseen: order Paid exactly once, one shipment intent, one audit row. Seen: existing result returned, nothing else changes.
executeRefund(refund)	Refund Approved (or Failed with prior approval).	Gateway success ⇒ Executed and order Closed; gateway failure ⇒ Failed, approval retained, order unchanged.

5.2 Validation scenarios: prediction first, then observation

Method: executable model (fulfiliq_test_model.py, Appendix A). The expected result and enforcing requirement were written first; the model's assertions encode them. **Observed** = result of running the model (output in Appendix B). **Not demonstrated** = needs the real application/database.

ID	Scenario	Predicted behaviour	Enforced by	Observed (model)	Still needed
T1	Two (here 20 threaded) customers buy the last unit	1 order; others "out of stock"; available = 0	FR04, INV3	PASS	DB-level concurrent test
T2	Duplicate callback ×10 while first processing (threaded)	1 payment, 1 intent, 1 audit row	FR06, INV1	PASS	Real gateway retry timing
T3	Approved refund fails at gateway, then retry	Failed → Executed, order Closed, 2 attempts	FR15, INV2	PASS	Gateway adapter test
T4	Dispatch without payment / without pick	Rejected, state unchanged (Paid)	FR08, FR10	PASS	API endpoint test
T5	Courier timeout (reply lost), then retry	Order stays Paid; 1 shipment only	FR11	PASS	Real courier adapter
T6	Support/customer approve refund; cross-customer read; execute unapproved	All rejected; refund stays Requested	FR01, FR14, INV2	PASS	Endpoint/role-matrix test
T7	Payment arrives after expiry, unit resold	Not Paid; flagged needs-refund; stock not oversold	FR07	PASS (after revision)	Real scheduler timing
T11	Client price tampering; qty 0	Rejected; total 236.00 for 2 units	FR02, FR03	PASS	Regional tax tables

5.3 Weakness found and design revision

Weakness (R1): a structured walkthrough of FR05 against FR06 showed that a callback arriving after the reservation was released could mark the order Paid although the unit had been resold, creating a paid order with no stock. **Revision:** on a late callback the system re-reserves if stock remains; otherwise it records the payment, flags "needs refund", and opens an approval-gated refund. FR07 was added, the state model (Cancelled → refund path) updated, and T7 now checks it. **Second weakness (I3):** the Failed

refund state had no retry path (see Q3); corrected and verified by T3. **Limitation:** the model is single-process; it does not prove database-level isolation, so a concurrent database test remains required.

Q6. Redesign for Expansion and Partial Failure

A. Impact analysis

Change	Affected requirements / assumptions	Affected components and qualities
3 → 12 regions; local tax, shipping, refund rules	FR03, FR14, A1, A3, UQ1; new FR18	Order, Regional policy, Refund; maintainability, correctness
50 → 500 users	NFR01, NFR02, NFR07	API replicas, DB connection pool; performance, availability
100 → 1,000 events/min (≈17/s average; peak assumed 5× ≈ 83/s)	NFR01, NFR03, FR19	Ingest path, Stock module, DB writes; consistency, throughput
Disconnected scanners	FR09, FR19, INV3	Scanner buffer, ingest dedup; availability vs staleness
Locally queued refund approval	FR14, FR15, FR20, INV2	Refund module, UI; accountability
Consolidated reports; limited budget	FR17, NFR07	Summary tables, optional read replica; cost

Figures are design assumptions, not demonstrated capacity.

B. Revised architecture, ownership, buffering and synchronization rules

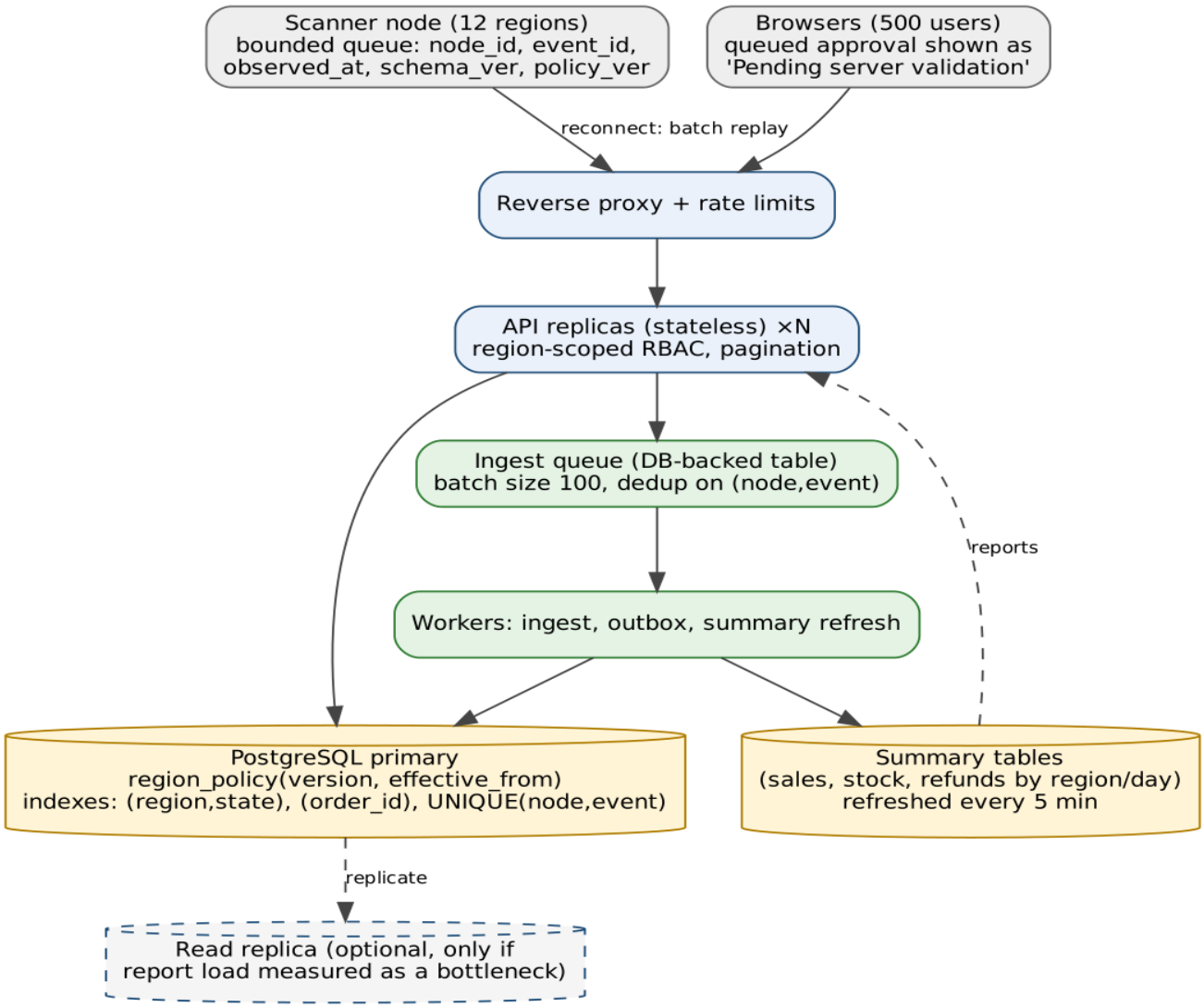


Figure 8. Revised architecture for 12 regions, buffered scanners and report summaries.

- **Regional policy:** tax, shipping and refund limits are versioned records (region, version, effective_from). Each order stores the policy_version used at pricing, so later rule changes never rewrite history. Order identity stays globally unique (UUID); RBAC permissions are scoped by region. **Data ownership:** Order module owns orders and policies; Stock module owns stock and reservations; Refund module owns approvals; Reporting owns only derived summary tables.
- **SYNC1 Event identity:** each scan/stock event carries node_id, event_id (unique per node), observed_at, schema_version and policy_version; the buffer is bounded (e.g. 5,000 events; when full the scanner blocks new scans and warns the picker rather than dropping silently).
- **SYNC2 Deduplication:** on reconnect events are sent in batches of 100; the server inserts under UNIQUE(node_id, event_id) before changing stock, so replays have no second effect.
- **SYNC3 Staleness:** events older than the configured age are marked stale and evaluated by server rules (e.g. flagged for stock correction by a human) instead of being presented as live counts.

- **SYNC4 Approvals:** an approval captured offline is stored locally as "Pending server validation"; the server re-checks authentication, role, limit and refund state, and only then moves it to Approved. The UI must never show Approved on the basis of a local queue (FR20).
- **Scaling justified by bottleneck, not by service count:** composite indexes (region, state) and (order_id) for the dominant queries (to be confirmed by query plans); pagination on all lists; a DB-backed ingest queue to absorb bursts; stateless API replication for CPU-bound request handling; **summary tables refreshed every 5 minutes** so reports do not compete with checkout. A second API replica alone does not remove a database or external-service bottleneck, so the single primary database is kept for now.

C. Additional validation scenarios

ID	Scenario	Expected behaviour	Evidence status	Result
S1 (T 8)	Scanner reconnects and replays the same 5 events (twice)	5 stock changes only; stock 10 → 5	Model: executed	PASS
S2 (T 9)	Refund approval queued offline	Refund stays Requested; local label "Pending"; execute rejected	Model: executed	PASS
S3 (T 10)	200 mixed operations incl. duplicate callbacks and expiries (burst)	Available stock never negative	Model: logic sweep only, not load	PASS
S4	One API replica fails during checkout load	Other replica serves; no data loss	Proposed; needs deployed environment	Not executed
S5	Regional policy changes while events are buffered	Event keeps its recorded policy_version	Walkthrough only	Gap: add policy_version to scan schema (done in SYNC1)

D. Defence of the design, trade-offs and limitations

- **Benefits:** one transactional core preserves the invariants; the outbox and the scanner buffer keep the system usable during courier, gateway or network failures; summary tables protect checkout from report load, at low cost.
- **Costs and consistency/availability trade-offs:** during a central database failure writes stop (consistency chosen over availability); offline scanning keeps the floor working but stock shown is delayed, and offline global approval is deliberately not allowed. Extra cost is limited to one more

API container, a worker and summary-table storage; a read replica or a broker is added only if measured evidence demands it.

- **Limitations:** 500 users and 1,000 events/min are planning targets; no load test has been run; the database is still a shared bottleneck and single point of failure; regional data-residency rules are not analysed.

Q7. Individual Technical Reflection

- **Requirement with greatest influence:** FR06 (idempotent payment confirmation). It forced the UNIQUE(gateway, event_id) constraint, the single-transaction design that led to choosing Alternative A, and the retry protocol in Figure 3.
- **Hardest conflict:** C4, fast refunds versus finance control. Customer satisfaction argues for speed while accountability argues for a gate; the compromise (request by anyone eligible, approval above a limit by finance only, execution separately) depends on an unresolved limit (UQ1).
- **Inconsistency found through modelling:** I3 (Failed refund without a retry edge) and I1 (browser success page treated as payment). Both appeared only when the sequence, state and scenario views were compared.
- **Unresolved trade-off:** centralized consistency is simple and correct, but it creates central-database dependency and limits availability. Splitting services (Alternative B) would reduce that dependency but add saga and operational complexity.
- **Evidence needed before implementation:** real query patterns and data volumes, gateway callback/retry behaviour, courier outage history, a concurrent database test for T1/T2, a load test at 50 and 500 users, a restore drill (NFR04), real refund limits per region (UQ1), and security review of the role matrix.

Appendix A. Code files and how to run

fulfiliq_test_model.py (delivered separately) – in-memory model of order, stock reservation, idempotent payment callback, lifecycle guards, outbox worker with reconciliation, refund approval/execution/retry, role and ownership checks and offline-scan deduplication, plus 11 checks T1–T11. Run: **python fulfiliq_test_model.py** (Python 3.9+, no external packages). A thread lock stands in for a database transaction. It uses synthetic data and test stubs for the courier and gateway.

Appendix B. Observed model output

PASS T1 Last unit, 20 threaded buyers | PASS T2 Duplicate callback x10 (threaded) | PASS T3 Refund fails at gateway, retry | PASS T4 Dispatch without payment/pick | PASS T5 Courier timeout then retry | PASS T6 Unauthorised approve / cross-read | PASS T7 Late payment after expiry | PASS T8 Offline scan replay (S1) | PASS T9 Queued approval not Approved (S2) | PASS T10 Invariant sweep, 200 ops (S3) | PASS T11 Price tampering / bad qty | 11/11 checks passed

Appendix C. Sources and tools acknowledgement

Tools: Python 3 (model), Graphviz and Mermaid CLI (diagrams), Jira/Confluence and GitHub as the reference tracking and documentation arrangement (selected, described in Q2; not live-hosted for this submission). Sources: the CSA10 Assignment 2 brief; standard concepts of UML, MoSCoW prioritization, idempotency keys, transactional outbox and optimistic locking. Stakeholder responses are simulated. Diagrams and code were generated for this submission with the assistance of an AI tool (Claude) and must be reviewed and understood by the student before submission.